

# Modulo 1: Nivel de aplicacion. Tema 2: Protocolo HTTP

## Informacion del protocolo

### Versiones

| Versión  | Año<br>Introducción | Características                                                                                                               |
|----------|---------------------|-------------------------------------------------------------------------------------------------------------------------------|
| HTTP/0.9 | 1990                | Versión inicial de HTTP, con soporte para un único verbo (GET) y contenido (HTML)                                             |
| HTTP/1.0 | 1996                | Soporte para navegadores, y nuevo contenido. Conexiones no-persistentes                                                       |
| HTTP/1.1 | 1997                | Sigue siendo el más utilizado. Introdujo conexiones persistentes y pipelining.                                                |
| HTTP/2   | 2015                | Protocolo binario, que hace un uso más extensivo de la compresión. Utilizado actualmente por aprox. el 40% de los servidores. |

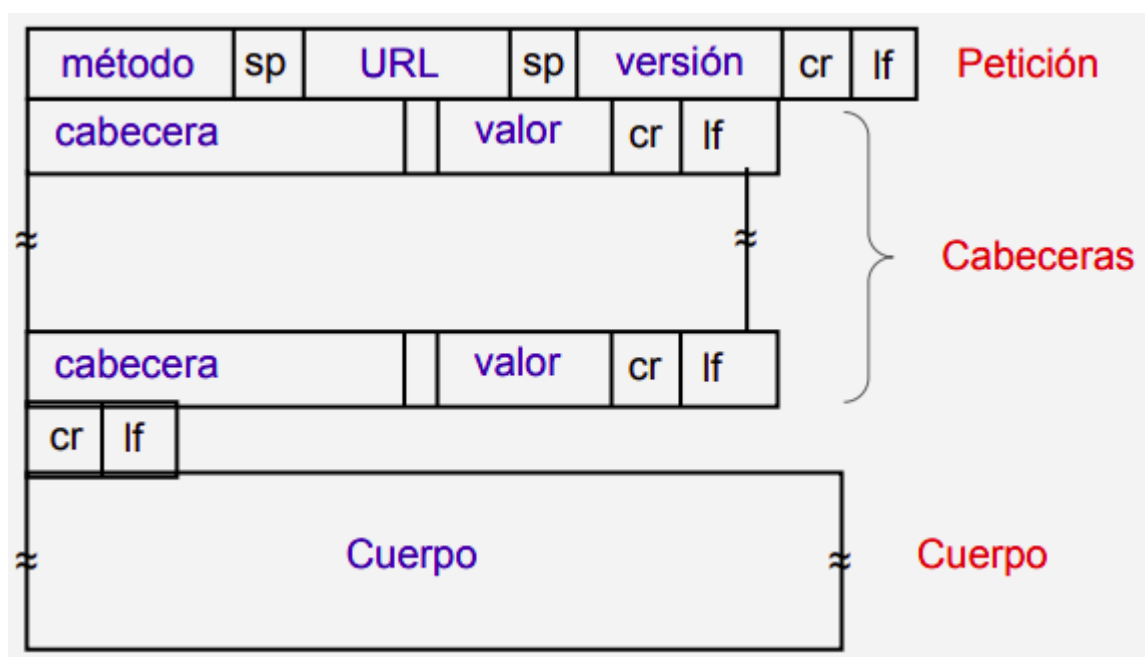
### Metodos

| Método                | Descripción                                                                                                                                                                                                                      |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET                   | Obtiene un recurso del servidor, aunque también puede enviar datos, codificando los parámetros en la propia URL. Esto puede ser inseguro (pues pueden quedar datos sensibles, como una contraseña, en los logs del servidor Web) |
| POST                  | Envía datos al servidor (p. ej., de un formulario), codificados en el cuerpo del mensaje                                                                                                                                         |
| HEAD                  | Obtiene solo las cabeceras de un recurso                                                                                                                                                                                         |
| OPTIONS               | Determina los métodos soportados por el servidor                                                                                                                                                                                 |
| PUT <sup>(*)</sup>    | Envía un fichero al servidor.                                                                                                                                                                                                    |
| DELETE <sup>(*)</sup> | Borra un recurso del servidor (si se tienen permisos para ello)                                                                                                                                                                  |

| Rango completo | Rango definido | Categoría         | Ejemplos                                                                                                                  |
|----------------|----------------|-------------------|---------------------------------------------------------------------------------------------------------------------------|
| 100-199        | 100-103        | Información       |                                                                                                                           |
| 200-299        | 200-208*       | Éxito             | 200 - OK [Recurso encontrado]                                                                                             |
| 300-399        | 300-308        | Redirección       | 301 - Moved permanently [El recurso ha sido movido a otra URL, y el navegador debe redireccionar a ella]                  |
| 400-499        | 400-431*       | Error de cliente  | 400 - Bad request [La petición no está bien formada]<br>404 - Not found [El recurso no se encuentra en la URL solicitada] |
| 500-599        | 500-511        | Error de servidor | 500 - Internal Server Error [El servidor sufre un error que no es culpa del cliente]                                      |

## Peticiones y respuestas HTTP

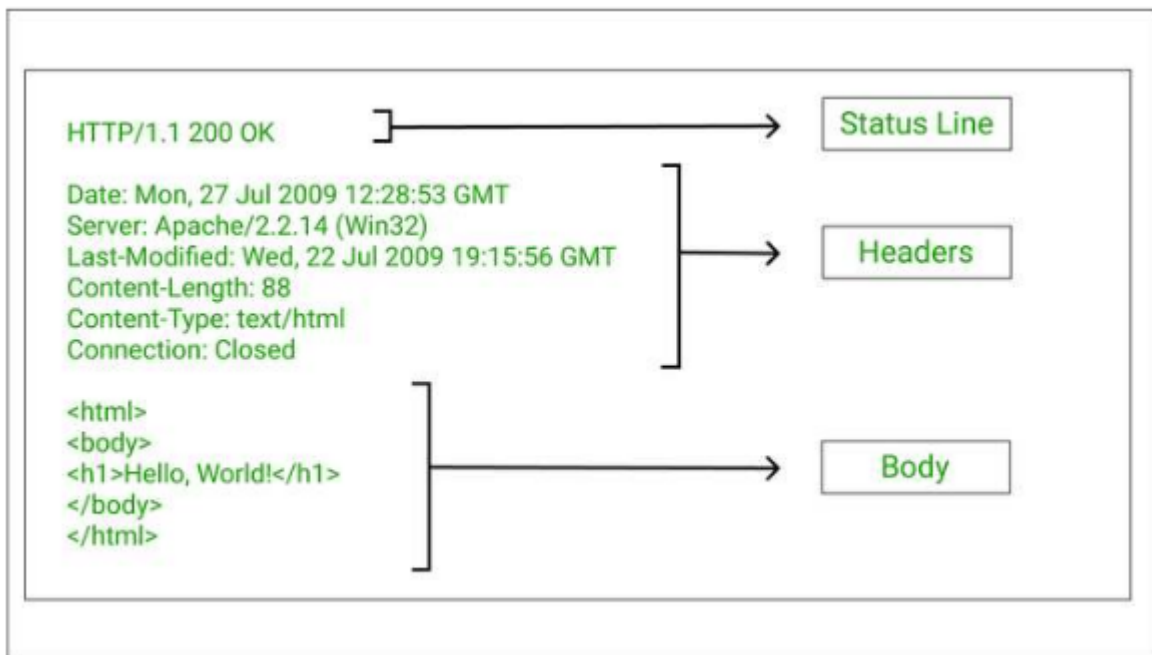
### Formato general de una petición HTTP



Se distinguen tres partes separadas por `\r\n`:

- Petición (Request Line): **[Operacion] [URI] HTTP/[Version]**
  - Operacion: de la lista de arriba (GET, POST, HEAD, ...)
  - URI: direccion del recurso en la que aplicar la operacion. Tambien se puede utilizar una URL (incluye la direccion de la web al principio)
  - Version: version de HTTP que quieres usar
- Cabeceras (Request Headers): **[Id]: [Valor]**, cada uno separado por un retorno de carro y un salto de linea. Son opcionales (casi todas) y dan informacion adicional sobre el cliente al servidor
  - ID: identificador de la cabecera
  - Valor: valor de esa cabecera
- Cuerpo (Request Message Body): no tiene sintaxis concreta. Da informacion de la pagina web, no del servidor. Se utiliza especialmente en las respuestas del servidor (ya que en el cuerpo se mete el recurso que habias pedido) y en formularios (para indicar al servidor que pone en cada campo)

## Formato de una respuesta HTTP



Estructurado de manera similar:

- Estado (Status Line): **HTTP/[Version] [Codigo] [Texto]**
  - Version: version de HTTP que quieres usar
  - Codigo: de la lista de arriba (200, 404, 500, ...)
  - Texto: texto breve explicando el código de respuesta
- Cabeceras (Headers): igual que las peticiones. Existen cabeceras solo para peticiones (como Accept) y solo para respuestas (como Accept-Ranges)
- Cuerpo (body): igual que en las peticiones

## ACT-T1-02 Conectando a un servidor Web

Netcat es una herramienta sencilla para realizar conexiones raw y permite enviar peticiones “a mano”. Nos conectamos al servidor con el siguiente comando:

```
nc -C -v uam.es 80.
```

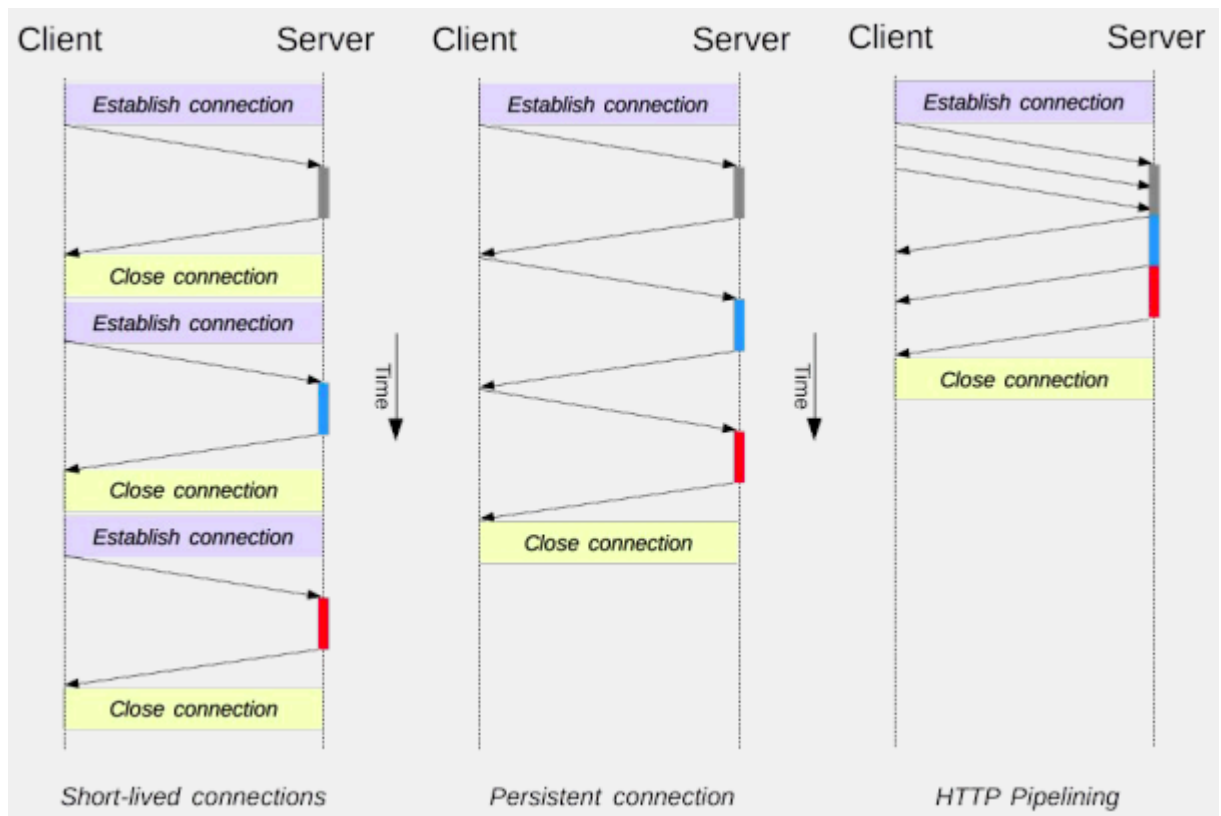
- -v: ‘verbose’, es decir, el comando va a hacer lo mismo, solo que te va a escribir en la terminal mas información de la normal.
- 80: puerto al que conectarse Enviaremos la petición HTTP: GET / HTTP/1.1

```
In [4]: !echo -e "GET / HTTP/1.1\r\nHost: uam.es\r\nConnection: close\r\n\r\n" | nc -C -v uam
```

```
Connection to uam.es (150.244.214.237) 80 port [tcp/http] succeeded!
HTTP/1.1 307 Moved Temporarily
Location: https://uam.es/
Content-Length: 0
```

Se puede ver que el servicio ha sido movido de localización, ya que ahora no se usa el protocolo http, se usa el https que es mas moderno y seguro

# Conexiones persistentes y no persistentes

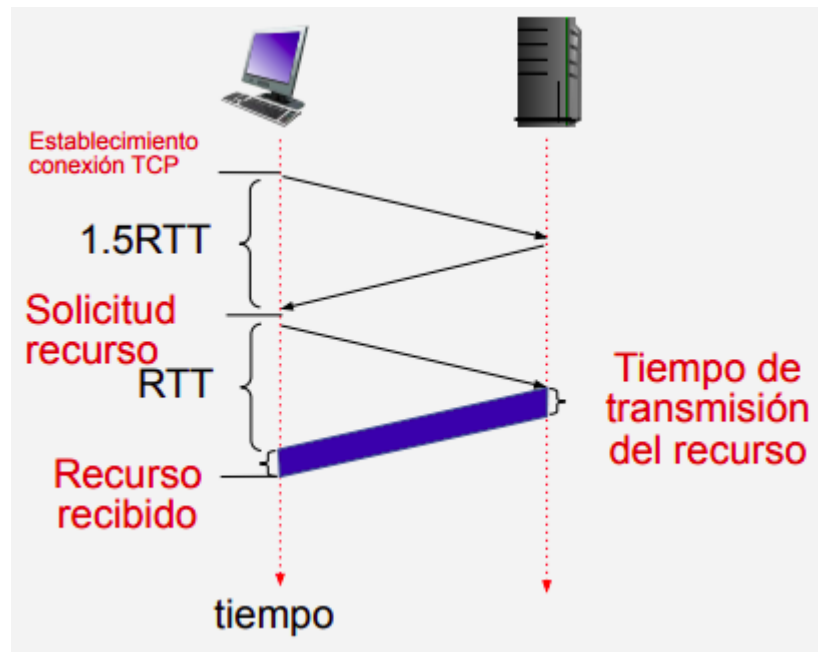


- **No persistente:** La conexión TCP se establece y se **cierra** para cada recurso que queremos sacar del servidor (html, imagenes, cookie, ...). Los navegadores abren multiples conexiones simultaneas. Es la opcion por defecto en HTTP/1.0
- **Persistente:** la conexión **no se cierra**, de forma que pueden enviarse varios objetos por una unica conexión TCP. Es la opcion por defecto en HTTP/1.1. Permite tecnicas como el pipelining.
- **Pipelining:** envio de peticiones aunque aun no se haya recibido la respuesta a la anterior. El servidor debe enviar las respuestas en el orden en el que se reciben, de manera que utiliza una cola FIFO. Si un paquete tarda en processarse en el servidor, bloquea toda la cola, ralentizando toda la conexión.

En ejercicios y modelos, consideramos un modelo lo mas realista posible del comportamiento de un navegador moderno (Chrome), a no ser que se inidque lo contrario:

- Cargan completamente una pagina Web, y solo despues, solicita los recursos que contiene
- Tiene un limite de 6 conexiones simultaneas con un mismo servidor, y 10 en total
- Si puede reutiliza conexiones abiertas

Ejemplo: Cuando entras a uam.es, el proceso es el siguiente:



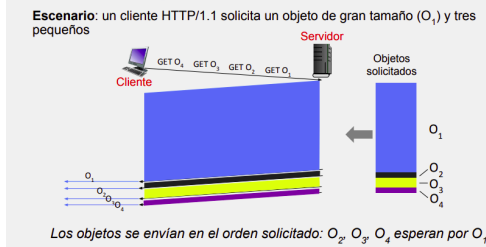
1. Conexión TCP con el servidor web de la UAM. Para este proceso se necesitan enviar 3 paquetes (explicación en los apuntes del tema 1). Teniendo en cuenta que RTT (Round-Trip Time) representa la ida y vuelta de un mensaje, se puede ver que este paso requiere 1,5 RTT.
2. Solicitud de la página web (el archivo html que queremos que el servidor nos envíe). Esto requiere mandar una petición al servidor y esperar su respuesta; es decir, 2 paquetes = 1 RTT. Sin embargo, HTTP se aprovecha del paso anterior: la petición la envía combinada con el tercer paquete del emparejamiento TCP.

Es decir, en el caso ideal (es decir, el archivo es pequeño) pedir un recurso nada más entrar a una página web requiere 2 RTT. En las diapositivas pone que 2,5 RTT. Eso está mal. En el libro de referencia (James F. Kurose, Keith W. Ross. (2010) Redes de computadoras. Un enfoque descendente. 7ª edición) lo explica (Figure 2.7). También lo he visto en este paper publicado en w3.org. Además, si la conexión es persistente, una vez realizada la conexión TCP puedes pedir el resto de recursos sin tener que repetir el paso 1; esto es, solo tardará 1 RTT.

## HTTP/2

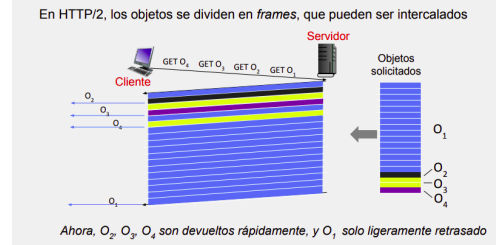
- **Compatible** con las versiones HTTP anteriores, por lo que la sintaxis y códigos se mantienen igual
- Una única conexión TCP
- Objetos con **prioridades** (entre 1 y 256), ya que hay elementos de una página que te interesa cargar antes que otros.
- Permite **Server push**, de manera que el servidor “anticipa” que vas a necesitar un recurso y te lo envía aunque no haya recibido aún la petición
- Los objetos se dividen en **frames** para mitigar el bloqueo HOL (bloqueo que ocurre con FCFS que los objetos pequeños tienen que esperar mucho si están detrás de objetos grandes)

## HTTP/2: mitigando el bloqueo HOL



Problema

## HTTP/2: multiplexing



Solución

## Manteniendo el estado

HTTP es un protocolo sin estado, es decir, en el servidor cada petición se trata de manera independiente sin establecer ninguna relación con las anteriores.

Una transacción Web normal necesita "recordar" quienes somos, para ello el servidor proporciona al navegador pequeñas piezas de información (cookie) únicas para cada usuario. Este las guarda y las vuelve a entregar al servidor cuando se visita el mismo dominio que generó la cookie.

### Cookies

Hoy en día, la herramienta más utilizada para informar al servidor de las peticiones anteriores es la Cookie. Consiste en un identificador único para tu ordenador en una página web. De esta manera, el servidor guarda un registro de todos estos identificadores que existen y así puede controlar la actividad de cada uno.

### Funcionamiento:



1. El cliente consulta si tiene una cookie para amazon.es. Como es la primera vez que se conecta a Amazon, no encuentra nada. Manda una petición HTTP habitual, es decir, sin cookie: `GET / HTTP/1.1\r\nHost: amazon.es`.
2. El servidor recibe esta petición. Antes de devolver el recurso solicitado (en este caso, la raíz de la página), ve que la petición no tiene la cabecera llamada **Cookie**. El servidor reserva espacio para el nuevo cliente (carrito de la compra, historial de compras, ...) y a esta memoria le da el **id 1678**.

Este identificador es nuestra cookie.

El servidor incluye en la respuesta HTTP la cabecera **Set-cookie: 1678**.

3. El cliente ve la cabecera **Set-cookie** y entiende que el servidor le ha asignado este identificador. De esta manera, en sus próximas peticiones se encargará de realizarlas con la cookie (utilizando la cabecera **Cookie: 1678**) para que el servidor pueda buscar en su base de datos, encontrar este id y devolverle contenido específico para él.

Por defecto, **la Cookie será eliminada al cerrar el navegador**. Sin embargo, el servidor puede cambiar su configuración y ponerle **fecha de caducidad**, de manera que sea válida hasta que llegue esa fecha.

## Tokens

Las aplicaciones Web necesitan autenticar y autorizar a sus usuarios

- **Autenticacion:** comprobar que un usuario es quien dice ser (típicamente, con usuario/contraseña)
  - **Autorizacion:** comprobar que un usuario autenticado tiene permisos para realizar cierta acción
- Cuando se utilizan para autenticar, las cookies deben comprobarse accediendo a la BD en cada petición del usuario → muy mal rendimiento. Los tokens no tienen este problema: cada token está firmado, lo que permite comprobar su validez sin más información. Se envía como una cabecera en cada petición: **Authorization: Bearer {Token}**

## JSON Web Token (JWT)

### Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MzE2MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

### Decoded

EDIT THE PAYLOAD AND SECRET

#### HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

#### PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

#### VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256-bit-secret
) ☐ secret base64 encoded
```

El string está compuesto de 3 partes (separadas por un punto en el string):

- **Header:** encabezado donde se indica, al menos, el algoritmo y el tipo de token, que en el caso del ejemplo anterior era el algoritmo HS256 y un token JWT.
- **Payload:** donde aparecen los datos de usuario y privilegios, así como toda la información que queramos añadir, todos los datos que creamos convenientes.
- **Signature:** una firma que nos permite verificar si el token es válido, y aquí es donde radica el quid de la cuestión, ya que si estamos tratando de hacer una comunicación segura entre partes y hemos visto que podemos coger cualquier token y ver su contenido con una herramienta sencilla, ¿dónde reside entonces la potencia de todo esto?

La firma se construye de tal forma que vamos a poder verificar que el remitente es quien dice ser, y que el mensaje no ha sido modificado por el camino. Se construye como el HMACSHA256, que

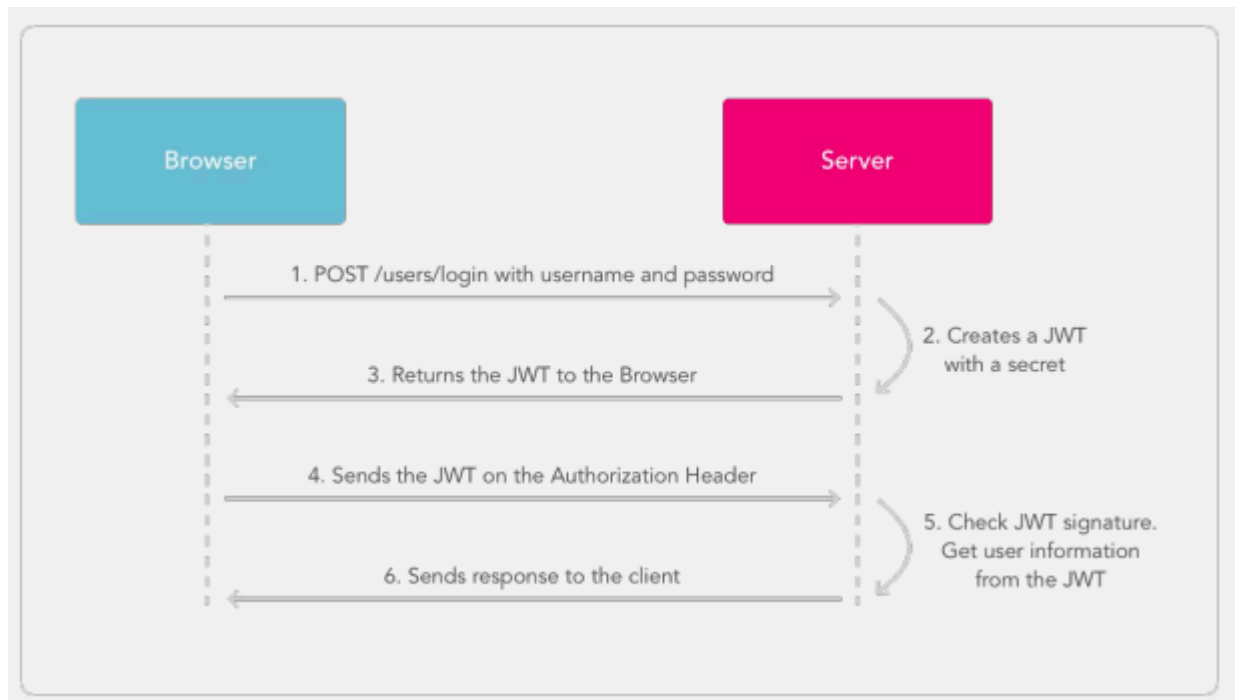


son las siglas de Hash-Based Message Authentication Code (Código de Autenticación de Mensajes), cifrado además con el algoritmo SHA de 256 bits. Se aplica esa función a:

- **Codificación en Base64 de header.**
- **Codificación en Base64 de payload.**
- **Un secreto**, establecido por la aplicación.

De esta forma, si alguien modifica el token por el camino, podríamos verificar que la comprobación de la firma no es correcta, por lo que no podemos confiar en el token recibido y deberíamos denegar la solicitud de recurso que nos haya realizado.

La interacción cliente-servidor usando JWT sería la siguiente:

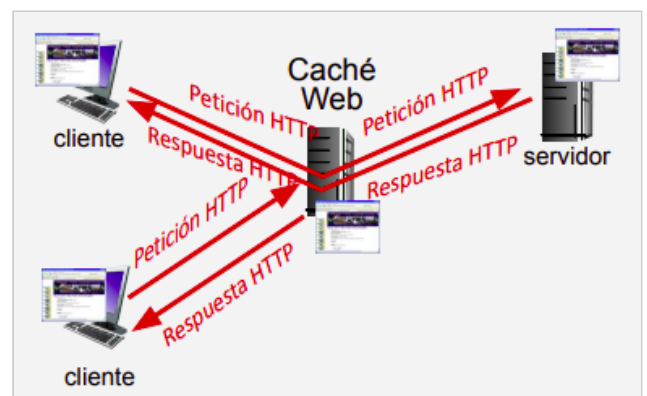


## Caches Web

**Objetivo:** evitar que las peticiones de un cliente tengan que viajar hasta el servidor de destino

El tráfico del usuario se configura para que viaje a través de un servidor cache Web local:

- si objeto en cache: se devuelve el objeto
- sino la cache pide el recurso al servidor, lo guarda para el futuro y devuelve al cliente



Esta solución reduce los tiempos de respuesta ya que la cache está más cerca (a menos saltos). El servidor decide qué contenido puede cachearse, influyendo en las cabeceras de las respuestas HTTP:

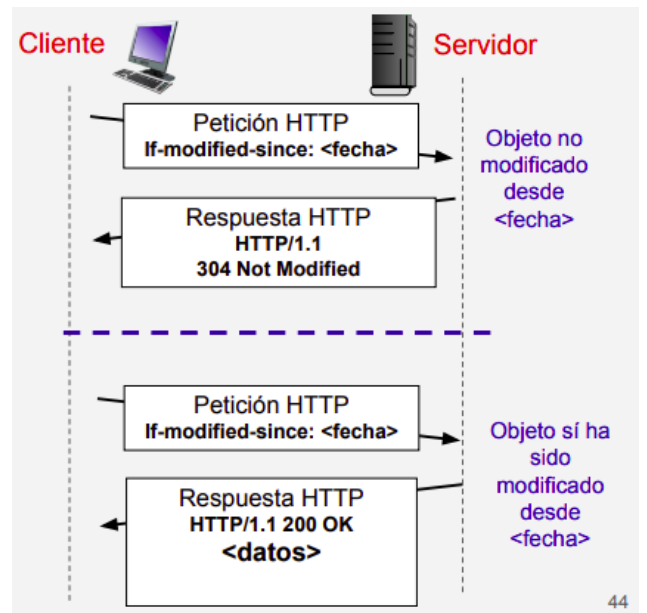
**Cache-Control: max-age=<seconds>** o **Cache-Control: no-cache**



## GET condicional

**Objetivo:** no realizar peticiones si el navegador tiene una versión actualizada del objeto solicitado

- **Cliente:** indica la fecha de la versión cacheada con cabecera específica en la petición HTTP: `If-modified-since: <fecha>`
- **Servidor:** si la copia está actualizada, no se devuelve el objeto: `HTTP/1.1 304 Not Modified`



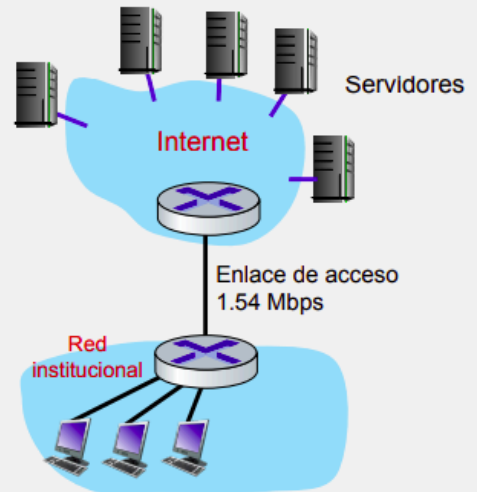
## Escenario real

### Escenario

- Ancho de banda del enlace de acceso: 1.54 Mbps
- RTT del router institucional a Internet: 2 seg
- Tamaño de los objetos: 100K bits
- Tasa medias de peticiones de los usuarios: 15 peticiones/seg
- **RESULTADO:** tráfico generado por los navegadores: 1.50 Mbps

### Rendimiento

- Utilización del enlace de acceso =  $1.5 / 1.54 = .97$
- Utilización LAN = .0015
- Tiempo total = Retraso Internet + Retraso enlace de acceso + Retraso LAN  
= 2 seg + minutos + usegs



## Opción: instalar una caché Web

Suponemos un tasa de acierto de la caché de 0.4:

- 40% de las peticiones se sirven por la caché, con muy poco retraso (mseg)
- 60% deben viajar hasta el servidor
  - Tráfico que atraviesa el enlace de acceso  
=  $0.6 * 1.50 \text{ Mbps} = .9 \text{ Mbps}$
  - Utilización del enlace =  $0.9 / 1.54 = .58$
- Retraso total:  
=  $0.6 * (\text{retraso Internet})$   
+  $0.4 * (\text{retraso de la caché})$   
=  $0.6 (2 \text{ segs}) + 0.4 (\sim \text{msecs}) = \sim 1.2 \text{ segs}$

